

Multimedia Technology Lab #7: Basic Image Manipulation and Processing in Python

1. Learning Objectives

1. Read, display, and write image files (BMP, JPEG) using Python.
2. Explain digital image representations under common Python libraries (skimage, and PIL)
3. Apply basic pixel-wise enhancement techniques (brightness/contrast adjustment, histogram equalization).
4. Apply image filtering and explain its basic principles
5. Explain the difference between lossless and lossy compression, and the trade-off between JPEG quality, file size, and image fidelity.

2. Required Software and Setup

Computers in the lab should be with Python 3.x installed. ทำตามขั้นตอนต่อไปนี้ เพื่อทำการสร้าง python virtual environment และ ทำการติดตั้ง software packages ที่จำเป็นสำหรับการทำแล็บ

- 1) เปิดหน้าจอ terminal (window command line) สร้าง folder ตั้งชื่อตาม student ID
- 2) เข้าไปที่ folder ที่ตั้ง ทำการพิมพ์คำสั่งต่อไปนี้เพื่อสร้างและเข้าสู่ python virtual environment (PS> คือ command prompt)

```
PS> python -m venv venv
PS> .\venv\Scripts\activate
```

- 3) ทำการติดตั้ง packages โดยรันคำสั่งต่อไปนี้

```
(venv)> python -m pip install numpy matplotlib pillow scikit-image
```

- 4) เข้าสู่โปรแกรม python และทำการตรวจสอบว่าได้ติดตั้ง package ต่าง ๆ ครบถ้วนไหม โดยการพิมพ์คำสั่งต่อไปนี้

```
import numpy, matplotlib, PIL, skimage
print("numpy:", numpy.__version__)
print("matplotlib:", matplotlib.__version__)
print("Pillow:", PIL.__version__)
print("scikit-image:", skimage.__version__)
```

Sample Images

รูปภาพที่จะใช้ในแล็บ จะเป็นไฟล์ที่มีมาพร้อมกับ package ‘scikit-image’. ให้ทดลองพิมพ์คำสั่งต่อไปนี้ เพื่อโหลดรูปภาพและแสดงผล

```
from skimage import data
import matplotlib.pyplot as plt #module for plotting and showing pictures

img = data.astronaut()          # colour image, 512x512
cam = data.camera()             # grayscale image, 512x512
plt.ion() #plt initialization so we can return to the REPL python prompt without closing picture
plt.imshow(img)
plt.title("Astronaut (sample image)")
plt.axis("off")
plt.show() #use this command to refresh
```

ทดลองแสดงภาพ cam ด้วยวิธีการข้างต้น สังเกตผลที่ได้ ทดลองแสดงภาพใหม่ ด้วยคำสั่ง `plt.imshow(cam, cmap='gray', vmin=0, vmax=255)`

Python 101

- Python เป็นภาษาโปรแกรมแบบ interpreter สามารถรัน code ที่ละบรรทัดใน REPL interactive environment ได้
- พิมพ์ python จาก command prompt เพื่อเปิด REPL interactive environment, พิมพ์ exit() เพื่อออก
- การใช้ tab ของ python มีความสำคัญมากสำหรับใช้กับ LOOP และ IF statement
- สามารถใช้การ copy & paste ที่ละหลาย ๆ บรรทัดเพื่อรัน code ได้ แต่ต้องระวังเรื่อง tab ตามข้อที่ผ่านมา แนะนำว่าถ้าจะทำการ copy ให้ทำกับ copy (เช่น จาก code ที่เขียนใน Visual Studio Code) ที่ละบรรทัด อย่า copy & paste ทั้งหมดมารันในคราวเดียวกัน เพราะหากมี error จะหาไม่เจอ หรือไม่สังเกต
- ตัวแปรประเภท List ใน python คือ array ที่สามารถระบุ element ใน list ได้ เช่น `arr = [1, 2, 3, 4]` จะระบุ สมาชิกตัวที่ 3 ให้พิมพ์ `arr[2]` ถ้าจะใช้ตัวที่ 2 และ 3 ให้พิมพ์ `arr[1:3]` -> `[2, 3]` (index เริ่มที่ 0) ถ้าจะอ้างอิง 2 ตัวแรกให้พิมพ์ `arr[:2]` หรือ `arr[0:2]`

3. Part 1 — Reading/Writing BMP / JPEG Files and Image Resizing

3.1 Background

- 1) BMP เป็นไฟล์ภาพ bitmap แบบดั้งเดิม ปกติไม่มีการบีบอัด (uncompressed or minimally compressed) — every pixel is stored explicitly, file sizes are large but no quality loss.
- 2) JPEG ใช้การบีบอัดแบบ lossy compression ผ่านกระบวนการ Discrete Cosine Transform (DCT) — it discards visual information that is less perceptible to the human eye to achieve much smaller file sizes.
- 3) ข้อมูลภาพสี จัดเก็บได้ทั้งในหลายรูปแบบเช่น RGB Model และ HSV Model
- 4) ในระดับล่างสุด (ระดับ data structure) รูปภาพที่จัดเก็บด้วย skimage module จะอาศัยการจัดเก็บข้อมูลด้วยโครงสร้าง (data structure) แบบ multidimensional array เช่น ภาพสี จะอาศัย array ที่มี 3 dimensions โดย dimension สุดท้ายเป็นแต่ละองค์ประกอบต่าง ๆ ตาม model สีที่ใช้ เช่น ถ้าเป็น RGB, สำหรับภาพที่เก็บในตัวแปร `org_arr`, `org_arr[:, :, 0]` คือภาพสีแดง, `org_arr[:, :, 1]` คือภาพสีเขียว เป็นต้น ในขณะที่ `org_arr[0:20, 0:10, 0]` คือส่วนของมุมบนซ้ายของภาพ ขนาด 20x10 pixels (สูง x กว้าง) สำหรับองค์ประกอบสีแดง

3.2 Using Pillow (PIL) and skimage

- 1) ทดลองพิมพ์และรัน code ต่อไปนี้ ซึ่งเป็นการอ่านรูปภาพจาก skimage module ซึ่งจัดเก็บเป็น n-dimensional array ทำการแปลงเป็นรูปแบบภาพแบบ PIL-Image และบันทึกภาพเป็น BMP และ JPEG จากนั้นอ่านไฟล์รูปภาพทั้งสอง เพื่อแสดงผล เปรียบเทียบภาพที่ได้จากไฟล์รูปภาพ JPEG และ BMP

```
from PIL import Image
from skimage import data
import numpy as np

# Create a BMP file from a sample image in skimage package
org_arr = data.astronaut()
img = Image.fromarray(org_arr) #conversion from array-format to Image format
img.save("astronaut.bmp")
img.save("astronaut.jpg", quality=90)

# Reopen and inspect
bmp_img = Image.open("astronaut.bmp")
jpg_img = Image.open("astronaut.jpg")
```

```
plt.figure(1) #specify the figure window for next image display
plt.imshow(bmp_img)
plt.figure(2)
plt.imshow(jpg_img)
```

2) ทำการตรวจสอบขนาดภาพ และ Color Model ที่ใช้ในการจัดเก็บภาพ

```
import type
print(type(img))    # <class 'PIL.Image.Image'>
print(img.size)     # (512, 512) -> Width, Height
print(img.mode)     # 'RGB'
```

3) ทำการบันทึกภาพ JPEG โดยเปลี่ยนค่า quality เป็นค่าต่าง ๆ ตามตัวอย่างต่อไปนี้ จากนั้นทำการเปรียบเทียบภาพ และตอบคำถามข้อ 1-2 ใน Lab-Quiz 7.3 (Onlearn) (ใช้เว้นขยายในหน้าต่างรูปภาพ เพื่อซูมเข้าไปดูตรงบริเวณใบหน้า สังเกตบริเวณพื้นหลัง และขอบรอยต่อระหว่างศีรษะและลำตัวกับพื้นหลัง)

```
img.save("astronaut_10.jpg", quality=10)
img.save("astronaut_50.jpg", quality=50)
img.save("astronaut_70.jpg", quality=70)
img.save("astronaut_90.jpg", quality=90)
```

```
# Reopen and inspect
# Use similar commands in the previous steps to open and view each image
# ex: jpg_90 = Image.open("astronaut_90.jpg")
```

4) ทำการแปลงข้อมูลภาพในรูปแบบ PIL-Image เป็น Array แล้วพิมพ์ค่าในตัวแปรใหม่บางส่วน เปรียบเทียบกับค่าจากข้อมูลต้นฉบับ (org_arr) ตามตัวอย่างต่อไปนี้

```
new_arr = np.array(jpg_90)
print(new_arr[0:10, 0:10, 0])
print(org_arr[0:10, 0:10, 0])
```

5) ทำแบบเดียวกัน โดยเปรียบเทียบต้นฉบับกับภาพที่ได้จากไฟล์ BMP นำผลที่ได้ไปตอบคำถาม Lab-Quiz 7.3 ข้อ 3

```
bmp_img = Image.open("astronaut.bmp")
new_arr2 = np.array(bmp_img)
print(new_arr2[0:10, 0:10, 0])
print(org_arr[0:10, 0:10, 0])
```

3.3 Image Data Structure

- 1) ในการทดลองที่ผ่านมา org_arr เป็นตัวแปรแบบ multidimensional array (n-array) ตามที่ระบุโดยโมดูล NumPy (**Numerical Python**. for fast math on large sets of numbers) ในหัวข้อนี้ ให้ลองทำการสร้างรูปภาพขึ้นมาใหม่ โดยการสร้างตัวแปร multidimensional array ตามตัวอย่างต่อไปนี้ ซึ่งจะแสดงประเภทข้อมูลที่จัดเก็บและขนาดของ array

```
import numpy as np #if already imported, no need to reimport it
# Create a 2x3 matrix (2 rows, 3 columns)
matrix = np.array([[50, 100, 50], [200, 70, 200]], dtype=np.uint8) #uint8 is unsigned integer 0-255
print("2D Matrix:")
print(matrix)

import type
print(type(matrix))    # <class 'numpy.ndarray'>
print(matrix.shape)    # (2,3)
```

```
print(matrix.dtype)    # uint8 or float64

#show data as image
plt.imshow(matrix, cmap= 'gray', vmin=0, vmax=255)
plt.title("Data as image")
plt.axis("off")
```

2) ทำให้เป็นภาพสีด้วย RGB Model โดยการปรับเปลี่ยนค่าในตัวแปร matrix ดังต่อไปนี้ แล้วแสดงผลเป็นภาพ

```
# Create a 2x3x3 matrix (2 rows, 3 columns, 3 color channels)
matrix = np.array([
    [[50, 0, 0], [100, 0, 0], [50, 0, 0]],
    [[200, 0, 0], [70, 0, 0], [200, 0, 0]]
], dtype=np.uint8)
print("3D Matrix:")
print(matrix)

import type
print(type(matrix))    # <class 'numpy.ndarray'>
print(matrix.shape)    # (2,3)
print(matrix.dtype)    # uint8 or float64

#show data as image
plt.imshow(matrix)
plt.title("Data as image")
plt.axis("off")
```

3) ทดลองปรับเปลี่ยนจากข้อที่แล้ว เพื่อให้ภาพเป็นสีเขียว และสีน้ำเงินตามลำดับ

4) ทำการจัดเก็บภาพในรูปแบบของ PIL-Image และดูรายละเอียดของภาพจากตัวแปรที่ได้ (สังเกตว่าจะไม่มีการระบุจำนวน color channel แต่จะปรากฏใน mode property ว่าเป็น การจัดเก็บด้วยโมเดล RGB)

```
pil_img = Image.fromarray(matrix)
print(type(pil_img))    # <class 'PIL.Image.Image'>
print(pil_img.size)     # (512, 512) -> Width, Height (No channel count here)
print(pil_img.mode)     # 'RGB'
```

5) เราสามารถอ่านและปรับเปลี่ยนค่าของแต่ละ pixel ใน object PIL-Image ได้ แต่ต้องทำการ load ไปยังตัวแปรที่เป็น array ปกติของ python ก่อนดังตัวอย่างต่อไปนี้ (สังเกตว่า index ตัวแรกคือ column ตัวที่ 2 คือ row ซึ่งจะสลับกับกรณีของ n-array ก่อนหน้านี้)

```
pixels = pil_img.load()
print(pixels[2, 0]) #print color value at pixel position col=2, row = 0
print(pixels[2, 0][0]) #print Red-color value at pixel position col=2, row = 0
```

6) ตอบคำถามข้อ 4 ใน Lab-Quiz 7.3

3.4 Image Resizing (Crop and scaling)

1) ทำการปรับขนาดของรูปภาพ (crop) ตามตัวอย่างต่อไปนี้

```
from skimage import data
import matplotlib.pyplot as plt

# Load the 512x512 astronaut image
img = data.astronaut()

# Crop the astronaut's face
# Rows 0 to 180, Columns 150 to 300
```

```
face_crop = img[0:180, 150:300]
```

```
# Display the cropped region
plt.imshow(face_crop)
plt.axis('off')
```

- 2) ทำการ scaling ให้ภาพมีความละเอียดลดลง (down sampling) ราว 2.5 เท่า โดยใช้วิธีการ nearest neighbor และ bilinear interpolation ตามตัวอย่างต่อไปนี้ (ดูให้แน่ใจว่าได้ import modules ที่จำเป็นครบถ้วน) เปรียบเทียบผลที่ได้ทั้ง 2 วิธีด้วยสายตา ว่ามีความแตกต่างกันหรือไม่ และเทียบคุณภาพกับภาพต้นฉบับ หลังจากเปรียบเทียบผลที่ได้แล้ว ให้ตอบคำถามข้อ 5 ใน **Lab-Quiz 7.3** (สังเกตบริเวณขอบถ้วย และ texture บนผิวไม้)

```
#downsampling with interpolation
import matplotlib.pyplot as plt
from skimage import data, transform

# 1. Load the sample image (Shape: 512, 512, 3)
img = data.coffee()

# 2. Define the new target dimensions (e.g., downsample by a factor of 2.5)
# 512 / 2.5 = 204.8 -> 205
target_shape = (205, 205)

# 3. Resize using nearest interpolation
# (order=0 is nearest-neighbor, order=1 specifies bilinear interpolation, order=3 is bicubic)
nearest_img = transform.resize(
    img,
    target_shape,
    order=0,
    anti_aliasing=False
)

# 4. Resize using bilinear interpolation
# (order=0 is nearest-neighbor, order=1 specifies bilinear interpolation, order=3 is bicubic)
bilinear_img = transform.resize(
    img,
    target_shape,
    order=1,
    anti_aliasing=False
)

# 5. Print shapes to verify the change
print("Original shape:", img.shape)          # (512, 512, 3)
print("Nearest-neighbor shape:", nearest_img.shape) # (128, 128, 3)
print("Bilinear interpolation shape:", bilinear_img.shape) # (128, 128, 3)

# 6. Display the result
plt.ion() #plt initialization so we can return to the REPL python prompt without closing picture
plt.figure(1)
plt.clf()
plt.imshow(img)
plt.title(f"Original {img.shape[:2]}")
plt.axis('off')

plt.figure(2)
plt.clf()
plt.imshow(nearest_img)
plt.title(f"Nearest-neighbor downsampled {nearest_img.shape[:2]}")
plt.axis('off')
```

```
plt.figure(3)
plt.clf()
plt.imshow(bilinear_img)
plt.title(f"Bilinear-interpolated downsampled {bilinear_img.shape[:2]}")
plt.axis('off')
plt.show()
```

- 7) ทำการทดลองโดยดัดแปลงจาก code ในข้อ 3 โดยกำหนด target_shape = (600, 900) เพื่อเพิ่มขนาดภาพ (พิกเซล) หรือ upsampling และทำการทดลองเพิ่มมาอีกหนึ่งวิธี ได้แก่ bicubic interpolation ตามตัวอย่างต่อไปนี้ เปรียบเทียบผลที่ได้ หากสังเกตไม่ชัด ให้แสดงเฉพาะบางส่วนด้วยคำสั่ง plt.imshow(bilinear_img[0:250, 150:360]) แทน (ทำเช่นเดียวกันสำหรับอีก 2 วิธีที่เหลือ) หลังจากเปรียบเทียบผลที่ได้แล้ว ให้ตอบคำถามข้อ 6 ใน [Lab-Quiz 7.3](#)

```
target_shape = (600, 900)
# Resize using nearest interpolation
# (order=0 is nearest-neighbor, order=1 specifies bilinear interpolation, order=3 is bicubic)
nearest_img = transform.resize(
    img,
    target_shape,
    order=0,
    anti_aliasing=False
)

# Resize using bilinear interpolation
bilinear_img = transform.resize(
    img,
    target_shape,
    order=1,
    anti_aliasing=False
)

# Resize using bicubic interpolation
bicubic_img = transform.resize(
    img,
    target_shape,
    order=3,
    anti_aliasing=False
)

plt.figure(1)
plt.imshow(nearest_img)
plt.title(f"Nearest-neighbor upsampled {nearest_img.shape[:2]}")
plt.axis('off')

plt.figure(2)
plt.imshow(bilinear_img)
plt.title(f"Bilinear-interpolated upsampled {bilinear_img.shape[:2]}")
plt.axis('off')
plt.show()

plt.figure(3)
plt.imshow(bicubic_img)
plt.title(f"Bicubic-interpolated upsampled {bicubic_img.shape[:2]}")
plt.axis('off')
plt.show()

#Zoom-in version (see instruction in a labsheet)
```

4. Part 2 — Image Enhancement

4.1 Brightness and Contrast

Brightness/contrast adjustment: a simple linear transform on pixel intensities: $\text{output} = \alpha * \text{input} + \beta$, where α controls contrast and β controls brightness. Values must be clipped to the valid $[0, 255]$ range.

- 1) ภาพ camera จากการทดลองแรก ๆ ในแล็บนี้ เป็นภาพขาวดำ เนื่องจากบุคคลในภาพใส่เสื้อโค้ดสีดำ ทำให้ภาพไม่สามารถแสดงรายละเอียดในบริเวณดังกล่าวได้ เราจะทำการทดลองเปรียบเทียบระหว่างการปรับเพิ่มค่าความสว่าง (brightness) และการปรับเพิ่ม contrast ของภาพดังกล่าว เพื่อให้เห็นรายละเอียดในบริเวณเสื้อดีขึ้น ให้ใช้ code ตัวอย่างต่อไปนี้ ในการปรับแต่งดังกล่าว สังเกตผลที่ได้ แล้วตอบคำถามข้อ 1-2 ใน Lab-Quiz 7.4 (ใน code .clip() ใช้จำกัดค่าที่ได้จากการคำนวณให้อยู่ในช่วงที่กำหนด ส่วน .astype(np.uint8) เป็น NumPy array method เพื่อแปลงประเภทข้อมูลให้เป็น uint8 – unsigned integer)

```
import numpy as np
import matplotlib.pyplot as plt
from skimage import data, util

# Load a sample image (uint8 array, values 0-255)
img_sk = data.camera()

# --- 1. Adjust Brightness (Addition) ---
# Add 40 to every pixel, then clamp between 0 and 255
bright_sk = np.clip(img_sk.astype(np.int32) + 40, 0, 255).astype(np.uint8)

# --- 2. Adjust Contrast (Multiplication) ---
# Multiply by 1.5 to scale intensities away from zero, then clamp
contrast_sk = np.clip(img_sk.astype(np.float32) * 1.5, 0, 255).astype(np.uint8)

# Display using matplotlib
plt.figure(1)
plt.clf() #clear current figure
plt.imshow(img_sk, cmap= 'gray', vmin=0, vmax=255)
plt.title("skimage Original")
plt.axis('off')
# use similar codes for displaying the other two images
```

- 2) ทำการพล็อตการกระจายค่าความสว่างของพิกเซลในภาพด้วยกราฟ Histogram โดยอาศัย code ตามตัวอย่างต่อไปนี้ เปรียบเทียบผลที่ได้ กับภาพที่แสดง

```
import matplotlib.pyplot as plt
from skimage import data
# 1. Open a new plotting figure window
plt.figure(1)
plt.clf() #clear current figure
# 2. Use .ravel() to flatten the 2D matrix into a 1D array
# bins=256 creates one bucket for every possible integer value
# range=(0, 256) locks the boundaries of the chart
img_sk_1d = img_sk.ravel()
plt.hist(img_sk_1d, bins=256, range=(0, 256), color='gray', alpha=0.75)
# 3. Label the axes and show the plot
plt.title("Pixel Intensity Histogram")
plt.xlabel("Pixel Intensity Value (0-255)")
```

```
plt.ylabel("Number of Pixels")
plt.grid(True, alpha=0.3)
plt.show()
```

plot a histograms of 'bright_sk' and contrast_sk using the above code for 'img_ks' as an example
use different figure numbers of each of them (ex: plt.figure(2), plt.figure(3))

- 3) ลองปรับเปลี่ยนรูปภาพโดยใช้ Standard Histogram equalization และ CLAHE (Contrast Limited Adaptive Histogram Equalization) เปรียบเทียบผลที่ได้ของ 2 วิธีดังกล่าว และเปรียบเทียบผลที่ได้จากการทำ Brightness และ contrast enhancement ในข้อที่ผ่านมา จากนั้น ทำการตอบคำถามข้อ 3-4 ใน [Lab-Quiz 7.4](#)

```
import matplotlib.pyplot as plt
from skimage import data, exposure, util

# Load a built-in sample grayscale image
img = data.camera()

#standard histogram equalization
equalized_img = exposure.equalize_hist(img)
equalized_imgu = util.img_as_ubyte(equalized_img) #convert floating point data to uint8

# Apply CLAHE natively using scikit-image
# kernel_size dictates the tile dimensions
clahe_img = exposure.equalize_adapthist(img, kernel_size=128)
clahe_imgu = util.img_as_ubyte(clahe_img) #convert floating point data to uint8

# Display the result
plt.figure(4)
plt.clf()
plt.imshow(img, cmap='gray')
plt.show(block=False)
plt.axis('off')

# Display the result
plt.figure(5)
plt.clf()
plt.title("Standard Equalization")
plt.imshow(equalized_imgu, cmap='gray')
plt.show(block=False)
plt.axis('off')

plt.figure(6)
plt.clf()
plt.title("CLAHE Equalization")
plt.imshow(clahe_imgu, cmap='gray')
plt.axis('off')
plt.show(block=False)
```

- 4) ลองใช้วิธีในการพล็อต Histogram ตามข้อที่ 2) กับภาพที่ได้จากขั้นตอนที่ 3) เปรียบเทียบความแตกต่างของกราฟที่ได้ (ใช้ภาพในตัวแปร equalized_imgu และ clahe_imgu) จะพบว่าวิธีหลังจะมีการกระจายตัวของค่าความสว่างที่ดีกว่า

4.2 Smoothing (spatial filtering)

- 1) ทำการอ่านรูปภาพ camera และแปลงให้เป็นข้อมูลแบบ floating point จากนั้น ทำการสร้าง filter kernel สังเกตค่าของ kernel ที่สร้าง โดยการรันคำสั่ง `print(kernel)`

```
#read camera data and set data type to float32 (so that it can perform floating point operations)
img = data.camera().astype(np.float32)
height, width = img.shape
#set kernel size
ksize = 3
padsz = int((ksize-1)/2)
#create a kernel (2D array) of size ksize x ksize
kernel = np.ones((ksize, ksize), dtype=np.float32) / (ksize*ksize)
print(kernel)
```

- 2) ทำการใช้ filter kernel ที่สร้างเพื่อทำการ filter ภาพ ตามตัวอย่างต่อไปนี้ สังเกตผลที่ได้

```
img_new = np.zeros((height-padsz*2, width-padsz*2), dtype=np.float32)
for i in range(0, height-padsz*2):
    for k in range(0, width-padsz*2):
        tmp = img[i:i+ksize, k:k+ksize] * kernel
        img_new[i, k] = np.sum(tmp)

#convert data back to uint8 type
img_new_uint8 = np.clip(img_new, 0, 255).astype(np.uint8)
plt.ion()
plt.clf()
plt.imshow(img_new_uint8, cmap='gray')
plt.axis('off')
```

- 3) ใช้ code ตามข้อ 1-2 แต่ปรับขนาดของ kernel (ksize) เป็น 5 แทน สังเกตภาพที่ได้ เปรียบเทียบกับผลที่ได้จากข้อ 2 ทำการตอบคำถามข้อ 5 ใน [Lab-Quiz 7.4](#)

5. Part 3 — Image Compression

- 1) ทดสอบทำความเข้าใจกระบวนการ lossy compression โดยใช้ DCT Transform โดยเริ่มจากการอ่านรูปภาพ และตัดภาพเฉพาะส่วน ขนาด 8x8 pixels ดังต่อไปนี้ ทำการแสดงผล block ที่ตัดออกมา

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.fftpack import dct, idct
from skimage import data, color

# 1. Load an image and crop a single 8x8 block of pixels
# JPEG compression processes images natively in 8x8 pixel blocks
img = data.camera()
# Extract an 8x8 block from a textured region (e.g., the coat/tripod)
block = img[300:308, 200:208].astype(np.float32)
plt.figure()
plt.imshow(block, cmap='gray')
```

- 2) ในกระบวนการ DCT Transform จำเป็นต้องปรับค่าความสว่างให้เป็น + และ - คือให้ค่าเฉลี่ยเป็น 0 จากนั้นจึงนำไปทำการแปลง DCT transform

```
# 2. JPEG Level Shift
# JPEG shifts pixels from [0, 255] to [-128, 127] to center the math around 0
```

```
shifted_block = block - 128.0
```

```
# 3. Apply 2D Discrete Cosine Transform (DCT)
# norm='ortho' scales the transform coefficients correctly
# type the following line exactly as shown. It is what required to do dct transform
# (.T = transpose -> a built-in function of an skimage object)
dct_block = dct(dct(shifted_block.T, norm='ortho').T, norm='ortho')
```

3) จาก DCT coefficients สามารถแปลงกลับเป็นภาพต้นฉบับ ได้ ตามตัวอย่างต่อไปนี้ (สังเกตว่าค่า DCT coefficient ไม่สามารถใช้แสดงผลเป็นรูปภาพได้โดยตรง ต้องทำกระบวนการย้อนกลับ idct ก่อน)

```
# Apply 2D Inverse DCT (IDCT)
idct_block = idct(idct(dct_block.T, norm='ortho').T, norm='ortho')
#
# Reverse Level Shift and Clip to valid uint8 space
reconstructed_block = np.clip(idct_block + 128.0, 0, 255).astype(np.uint8)

# =====
# PRINT AND VISUALIZE DATA MATRICES
# =====
np.set_printoptions(precision=1, suppress=True)

print("--- 1. ORIGINAL PIXEL BLOCK ---")
print(block.astype(np.uint8))

print("\n--- 2. RAW DCT COEFFICIENTS ---")
print("Top-left is DC (average brightness), others are spatial AC frequencies.")
print(dct_block)

# Plotting the visualization
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].imshow(block, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original 8x8 Block")
axes[1].imshow(np.log(np.abs(dct_block) + 1), cmap='gray')
axes[1].set_title("DCT Coefficients (Log Scale)")
axes[2].imshow(reconstructed_block, cmap='gray', vmin=0, vmax=255)
axes[2].set_title("JPEG Reconstructed Block")
for ax in axes:
    ax.axis('off')
#
```

4) ทำการคำนวณค่าพิเศษของ DCT Coefficients ด้วยคำสั่ง `np.round(dct_block)` จากนั้นคำนวณว่าหากจะจัดเก็บรูปภาพโดยใช้การจัดเก็บ DCT coefficients แทนการจัดเก็บค่าความสว่างของแต่ละพิกเซลโดยตรง (ซึ่งต้องใช้ข้อมูล 8 bits ต่อพิกเซลสำหรับภาพที่มีค่าความสว่างได้ตั้งแต่ 0 ถึง 255) โดยการหาค่าใน `dct_block` ค่า coefficient ใดที่เมื่อปัดเศษเป็นจำนวนเต็มแล้ว มีขนาด (ไม่นับเครื่องหมายว่าเป็นบวกหรือลบ) สูงสุด นำค่าดังกล่าวมาคิดว่าต้องใช้ข้อมูลกี่บิตจึงจะแทนค่าดังกล่าวได้ เช่น n bits จะได้จำนวนบิตที่ต้องใช้ในการจัดเก็บแต่ละค่า DCT coefficient เท่ากับ $n+1$ (ต้องบวกอีก 1 บิตสำหรับใช้แทนเครื่องหมายว่าเป็นบวกหรือลบ) (ตัวอย่างเช่น ถ้าค่า DCT coefficient ขนาดสูงสุด คือ -100.5 ปัดเศษแล้วเป็น -101 จำนวนบิตที่ต้องใช้คือ $n = 7 \Rightarrow n+1 = 8$ บิต ต่อ 1 ค่า) เมื่อคำนวณได้แล้ว ทำการตอบคำถามข้อ 1-3 ใน **Lab-Quiz-7-5**.

5) จากค่า dct coefficients (`dct_block` ที่ยังไม่ปัดเศษ) ให้นำไปผ่านกระบวนการ quantization ดังตัวอย่างต่อไปนี้ ทำการเปรียบเทียบค่า dct coefficients ก่อนและหลังการทำ quantization

```
# Standard JPEG Luminance Quantization Matrix (Quality Level ~50)
# Higher numbers in the bottom-right reject high-frequency details
Q = np.array([[16, 11, 10, 16, 24, 40, 51, 61],
```

```
[12, 12, 14, 19, 26, 58, 60, 55],
[14, 13, 16, 24, 40, 57, 69, 56],
[14, 17, 22, 29, 51, 87, 80, 62],
[18, 22, 37, 56, 68, 109, 103, 77],
[24, 35, 55, 64, 81, 104, 113, 92],
[49, 64, 78, 87, 103, 121, 120, 101],
[72, 92, 95, 98, 112, 100, 103, 99]], dtype=np.float32)
```

```
# Quantization (The "Lossy" Step)
# Divide by the matrix and round to the nearest whole integer.
# This forces many high-frequency AC coefficients in the bottom right to zero!
quantized_block = np.round(dct_block / Q)
```

- 6) ทำการแปลงค่า `quantized_block` กลับเป็นรูปภาพ เพื่อแสดงผลที่ได้ เปรียบเทียบภาพต้นฉบับกับภาพที่ได้จากการระบวนการ reconstruction ตอบคำถามข้อ 4 ใน [Lab-Quiz-7-5](#).

```
# =====
# DECOMPRESSION PIPELINE (Reconstruction)
# =====

# Dequantization
# Multiply back by the quantization matrix to restore rough values
dequantized_block = quantized_block * Q

# Apply 2D Inverse DCT (IDCT)
idct_block = idct(idct(dequantized_block.T, norm='ortho').T, norm='ortho')

# 8. Reverse Level Shift and Clip to valid uint8 space
reconstructed_block = np.clip(idct_block + 128.0, 0, 255).astype(np.uint8)

# =====
# PRINT AND VISUALIZE DATA MATRICES
# =====
np.set_printoptions(precision=1, suppress=True)

print("--- 1. ORIGINAL PIXEL BLOCK ---")
print(block.astype(np.uint8))

print("\n--- 2. RAW DCT COEFFICIENTS ---")
print("Top-left is DC (average brightness), others are spatial AC frequencies.")
print(dct_block)

print("\n--- 3. QUANTIZED DCT BLOCK (Compressed) ---")
print("Notice how many numbers smoothly drop to zero!")
print(quantized_block.astype(np.int32))

# Plotting the visualization
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
axes[0].imshow(block, cmap='gray', vmin=0, vmax=255)
axes[0].set_title("Original 8x8 Block")
axes[1].imshow(np.log(np.abs(quantized_block) + 1), cmap='gray')
axes[1].set_title("DCT Coefficients (Log Scale)")
axes[2].imshow(reconstructed_block, cmap='gray', vmin=0, vmax=255)
axes[2].set_title("JPEG Reconstructed Block")
for ax in axes:
    ax.axis('off')
#
```

```
plt.show()
```

- 7) ทำการคำนวณว่าหากจะจัดเก็บรูปภาพโดยใช้การจัดเก็บ DCT coefficients แทนการจัดเก็บค่าความสว่างของแต่ละพิกเซลโดยตรง ด้วยวิธีการเดียวกับข้อ 4) แต่เปลี่ยนจากค่าใน dct_block เป็นค่าใน quantized_block แทน นำค่าที่ได้ไปตอบคำถามข้อ 5 ใน Lab-Quiz-7-5.
- 8) หากในการจัดเก็บค่า dct coefficients ตามข้อ 7) ทำเฉพาะกับ coefficients ที่มีค่าไม่เป็นศูนย์เท่านั้น จะต้องใช้จำนวนบิตทั้งหมดเท่าใดในการจัดเก็บค่า dct coefficients ของบล็อกดังกล่าว เปรียบเทียบกับจำนวนบิตที่ต้องใช้ในการจัดเก็บค่าความสว่างของพิกเซลโดยตรง (ที่ต้องใช้ทั้งหมด 8 บิต x 64 พิกเซล = 512 บิต) นำค่าที่ได้ไปตอบคำถามข้อ 6-7 ใน Lab-Quiz-7-5.

6. Homework

- 1) เลือกภาพสีของนักศึกษา มา 1 รูป ขนาดไฟล์เมื่อ save เป็น JPEG ไม่เกิน 1MB (หากไฟล์ใหญ่เกินไปให้ ลดทอนขนาดภาพโดยการ crop หรือ downsampling)
- 2) นำภาพดังกล่าวมาทำการปรับความสว่างโดยใช้ วิธีการ CLAHE (Contrast Limited Adaptive Histogram Equalization) ตามตัวอย่างใน labsheet ข้างต้น แต่ต้องมีการปรับเปลี่ยนบางประการ เพื่อให้ใช้งานได้กับภาพสี โดยแทนที่จะทำ Histogram Equalization กับ สีแต่ละสี (R, G, B) ให้ทำการแปลงข้อมูลภาพที่จัดเก็บในรูปแบบ RGB ไปเป็น HSV ก่อนตามตัวอย่างต่อไปนี้

```
import matplotlib.pyplot as plt
from skimage import color, exposure, io

# 1. Load your color image
# (skimage loads images into standard RGB NumPy arrays)
img = io.imread("color_image.jpg")

# 2. Convert from RGB to HSV color space
# Output arrays will be floating-point numbers from 0.0 to 1.0
hsv_img = color.rgb2hsv(img)

# 3. Apply CLAHE only to the third channel (index 2: 'Value' / Brightness)
# kernel_size dictates the tile dimensions
#kernel_size = 64
hsv_img[:, :, 2] = exposure.equalize_adapthist(hsv_img[:, :, 2], kernel_size)

# 4. Convert back from HSV to standard RGB
output_rgb = color.hsv2rgb(hsv_img)

# 5. Display the result
plt.imshow(output_rgb)
plt.axis("off")
plt.show()
```

ให้ทำการทดลองกับค่า kernel_size ที่แตกต่างกัน 3 ค่า

- 3) เลือกค่าที่ให้ภาพที่ดีที่สุด ทำการเพิ่มความคมชัดในภาพ ด้วยวิธีการต่อไปนี้ (ให้เปลี่ยนข้อความตามชื่อ รหัส และ kernel size ที่ใช้ ตลอดจนปรับเปลี่ยนตำแหน่งของข้อความ ให้แสดงผลบริเวณด้านล่างของภาพ และกำหนดสีของข้อความให้เหมาะสม)

```
# 1. If your image is stored using skimage, you need to convert to PIL Image first
#
from PIL import Image, ImageDraw, ImageFont
```

```
# 2. Create an ImageDraw object
draw = ImageDraw.Draw(img) #img must be PIL Image object

# 3. Choose coordinates (X/Column, Y/Row)
text_position = (50, 100) #Please choose your text_position
#set the content to add. Make change to the sample below
text_content = "Name: Somchai, Student ID:666666, kernel size= 64"
text_color = (255, 255, 255) # Change to your color

# 4. Draw the text (using the default system font)
draw.text(text_position, text_content, fill=text_color)

# Optional: To use a custom font with a specific size
# custom_font = ImageFont.truetype("arial.ttf", size=36)
# draw.text(text_position, text_content, fill=text_color, font=custom_font)

# 5. Save or show the result
img.show()
img.save("68xxx_kernel_64.jpg")
```

- 4) ทำการบันทึกภาพลงเป็นไฟล์ jpeg โดยตั้งชื่อตามรหัสนักศึกษาและ kernel size จากนั้นบันทึกไฟล์โปรแกรม (code ที่ใช้) ทั้งหมดลงเป็นไฟล์เดียวกัน (ทั้งที่ใช้อ่านภาพ ทำ histogram equalization และที่ใช้เพิ่มข้อความ)
- 5) ทำการส่งงาน โดยแนบไฟล์ โปรแกรม, รูปภาพต้นฉบับ และรูปภาพที่ได้จากข้อ 4) ทำการส่งงานภายในก่อนเที่ยงคืนของวันถัดจากวันที่ทำแล็บ ทาง Google Form: <https://forms.gle/h7sCWfQcKjqTGkG66> (login ด้วย KMITL student account)

7. Submission Requirements

- 1) ได้เข้าเรียนแล็บตาม section ที่ลงทะเบียนเรียน และตอบคำถาม Lab-Quiz ครบภายในวันดังกล่าว ยกเว้น นศ ที่มีภารกิจติดเรียน หรือเป็น TA ที่ได้แจ้งไว้แล้ว อนุโลม ให้เข้าต่าง section จากที่ลงทะเบียนไว้ได้
- 2) ทำการทดลองเพิ่มเติมตามที่ระบุในหัวข้อ Homework ส่งผ่านทาง Google Form (ภายในวันถัดไป:ก่อนเที่ยงคืน) (เฉพาะคนที่เข้าแล็บตามเวลาที่ลงทะเบียนเท่านั้น ที่จะได้รับการพิจารณาคะแนน ยกเว้นที่ติดภารกิจ เช่น TA ซึ่งจะพิจารณาเป็นราย ๆ ไป)